

Forecasting weekly German electricity demand

7PAM2016-0509-2025 - Advanced Research Topics in Data Science

GitHub Link: <https://github.com/srihari4420/electricity-demand-forecasting>

Student Id: 24065408

1. Introduction

This report is about forecasting German electricity demand. The data are hourly load figures from Open Power System Data, covering January 2015 to October 2020. The hourly series was cleaned and averaged up to weekly demand in gigawatts, and eleven forecasting methods were compared on it, from very simple benchmarks up to a SARIMAX model with weather and holiday inputs, four regression and tree models, a Bayesian regression, and an LSTM neural network. Every model was tested on the same held-out period, the final 104 weeks, and judged against a seasonal naive forecast.

The main result is simple to state. The best model was also one of the simplest: a plain linear regression on a carefully built feature table, with a test RMSE of 1.73 GW, about 42% better than the seasonal naive benchmark. The Bayesian regression came second. The tree models beat the benchmark by a smaller margin. The two most complex models, SARIMAX and the LSTM, both did worse than the benchmark. Extra complexity was not rewarded on this problem, and most of this report is about why.

The whole analysis can be re-run from the code repository with one command, `python scripts/run_pipeline.py`.

2. Data and preparation

The target series is the ENTSO-E actual total load for Germany, taken from the Open Power System Data 60-minute file. From January 2015 the hourly record is essentially complete; the few short gaps were filled by time interpolation after putting the series on a regular hourly grid. The hourly data were then averaged to weekly demand and converted from megawatts to gigawatts, giving 301 weekly values of the target, `load_gw`.

Two choices are worth explaining. The timestamps are in UTC; the timezone was dropped, since at a weekly average a one-hour offset makes no real difference and it simplified the weather merge. And the split is strictly by time: the last 104 weeks are the test set, and nothing after that boundary is used to fit any model, because a random split would let the models see the future.

Daily mean temperature for Berlin was taken from the Open-Meteo archive as a stand-in for national weather, and turned into weekly features: mean, minimum and maximum temperature, plus heating and cooling degree-days, which measure how cold or hot each week was. German public holidays were counted per week, giving `holiday_days` and a `has_holiday` flag. One difference between these two inputs matters throughout: holidays are

known far in advance because the calendar is fixed, while future temperature is not known when a forecast is made.

3. Exploratory analysis

One pattern dominates the weekly series: a strong yearly cycle, with demand high in winter and low in summer. An STL decomposition (Figure 1) shows how large it is. The seasonal part swings far more than the trend, which is mild and mostly flat, or the leftover noise. There is also a clear drop in early 2020, matching the first pandemic lockdown. That drop sits inside the test window and turns out to matter a lot for the model comparison.

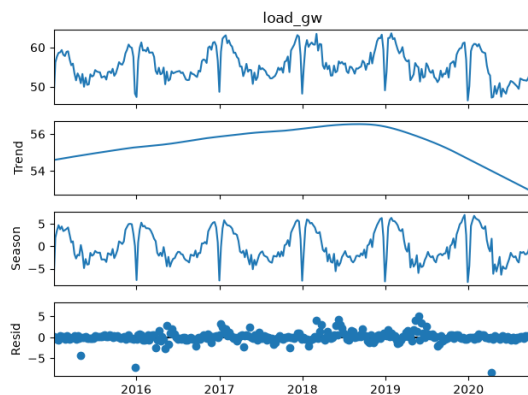


Figure 1. STL decomposition of the weekly series into trend, seasonal and remainder (GW). The seasonal part is much larger than the other components, so the yearly cycle is the main structure in the data.

Stationarity was checked with two tests that ask opposite questions. ADF rejects a unit root ($p = 0.001$) and KPSS does not reject stationarity ($p = 0.10$), so both agree the weekly level is broadly stationary. What the tests do not show, but the ACF does, is the seasonal pattern: slow decay, with clear structure reaching towards lag 52. One seasonal difference at lag 52 keeps ADF happy ($p = 0.0005$) but makes KPSS reject ($p = 0.01$), the usual sign of slight over-differencing. The practical lesson, used later for SARIMAX, is that differencing here is for modelling the seasonality, not for removing a unit root the tests say does not exist.

The weather inputs behave as expected. Weekly load correlates at +0.69 with heating degree-days and -0.64 with mean temperature, while cooling degree-days correlate at only -0.16. Germany is a heating country: cold weeks push demand up far more than hot weeks do. The weeks with public holidays, above all Christmas and New Year, show the deepest demand dips in the whole series, visible as the sharp yearly notches in the figures below.

4. The models

Each model family is described briefly here, along with the reason it was included.

Benchmarks. Four simple reference forecasts: the mean (training average every week), the naive (last training value repeated), the seasonal naive (last year's 52 weekly values replayed), and the drift method (a straight line through the first and last training points,

extended). Only the seasonal naive is a serious contender: with a yearly cycle this regular, copying last year is a strong forecast, and its test RMSE of 3.01 GW is the bar every other model has to beat.

SARIMAX. A seasonal ARIMA model with external inputs. It combines autoregression (this week depends on recent weeks), differencing (to handle trend and seasonality), and moving-average terms (to model recent shocks), with a seasonal copy of each at lag 52. The specification used was (1, 1, 1)(1, 1, 1, 52), with heating degree-days, cooling degree-days and holiday counts passed in as external regressors. Because the temperature values used in the test period are the observed ones, this is a conditional forecast rather than a true operational one, a point returned to below.

Feature-based models. The same problem rebuilt as standard supervised learning. A feature table was made from lagged load (1, 2, 4, 8, 13, 26 and 52 weeks back), rolling averages (4, 13 and 52 weeks), week-of-year and month, three sine-cosine pairs to describe the yearly cycle smoothly, holiday counts, and the temperature features with one-week lags. Four models were fitted on it with fixed random seeds. Linear regression fits one coefficient per feature and predicts with a weighted sum. Random Forest builds hundreds of decision trees on random subsets of the data and averages their predictions, which reduces variance. Gradient boosting builds trees one after another, each one correcting the errors of the ones before it. Histogram gradient boosting is a faster version of the same idea that groups feature values into bins.

Bayesian regression. A Bayesian ridge regression on standardised versions of the same features. It is a linear model with a prior belief that coefficients are small, and it learns how strong that belief should be from the data. What it adds over plain regression is uncertainty: every coefficient comes with a spread, and every forecast comes with a predictive standard deviation, which gives proper 95% intervals with no extra machinery.

LSTM. A long short-term memory network (Hochreiter and Schmidhuber, 1997), the standard neural network for sequences. The load-forecasting literature is fairly consistent about where it works: LSTMs do well on short-term, high-frequency data with lots of training examples, such as the smart-meter work of Kong et al. (2019) and the building-load study of Marino, Amarasinghe and Manic (2016), but their edge shrinks against simpler models with good features, and they need a lot of data to avoid overfitting. That sets the expectation here: 197 weekly training points is a very small sample for a neural network. The version used has a 52-week lookback, one LSTM layer of 32 units, dropout of 0.1 and a dense output, trained with Adam and early stopping, with scaling fitted on the training data only. The forecast is made recursively: predict one week, slide the window onto that prediction, repeat for all 104 test weeks. This is the honest way to forecast a long horizon, since feeding the network real values week by week would give it an easier task than the other models face. A longer lookback and a wider layer were also tried and made no real difference, which suggests the limit is the data, not the design.

How leakage was avoided

One rule runs through all the feature building, and it answers the leakage question directly. Every lag is a backward shift, so a row only ever contains information from earlier weeks. Every rolling average is computed on load shifted back one week before the window is applied, so the current week never appears among its own predictors. The temperature lags are shifted the same way. Anything that learns from the data, such as the scaling used by the Bayesian model and the LSTM, is fitted on the training set only and then applied to the test set. Without these steps the models would quietly use information that is not available at forecast time, and the test scores would look far better than they deserve.

5. How the models were tested

All models were scored on the same final 104 weeks using four metrics. MAE is the average size of the error. RMSE is similar but punishes big misses more. MASE is the MAE divided by the in-sample seasonal naive error, so a value below 1 means the model beats a seasonal naive guess. Bias is the average of forecast minus actual, so a positive value means the model forecasts too high. The last column of Table 1 shows each model's RMSE as a percentage change against the seasonal naive, which is the comparison the whole study rests on.

6. Results

Table 1 shows the test-window metrics for every model, sorted by RMSE, exactly as the pipeline produced them.

Model	MAE (GW)	RMSE (GW)	MASE	Bias (GW)	vs seasonal naive
Linear regression	1.262	1.733	0.943	+0.758	-42.4%
Bayesian ridge	1.361	1.883	1.017	+0.995	-37.4%
Hist. gradient boosting	1.772	2.399	1.324	+1.200	-20.2%
Gradient boosting	1.937	2.522	1.448	+1.166	-16.1%
Random Forest	1.904	2.540	1.423	+1.030	-15.5%
Seasonal naive (benchmark)	2.319	3.007	1.732	+1.732	0.0%
SARIMAX	3.018	3.792	2.255	+2.934	+26.1%
Mean	3.789	4.397	2.831	+0.481	+46.2%
LSTM (neural)	3.844	4.458	2.872	+0.879	+48.3%
Naive	3.783	4.459	2.827	-0.882	+48.3%
Drift	4.340	5.118	3.243	+1.007	+70.2%

Table 1. Test performance over the final 104 weeks. Only the models above the benchmark row give a meaningful improvement.

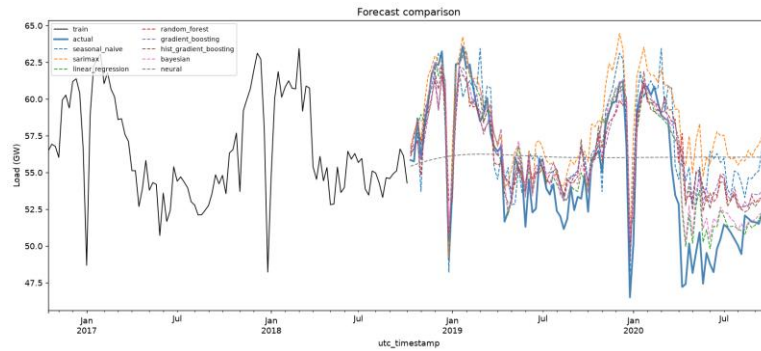


Figure 2. All model forecasts against the actual final two years. The linear regression (green) and Bayesian (pink) forecasts follow the actual line most closely, including the 2020 drop; the tree models sit just above them; SARIMAX (orange) drifts high from spring 2020; the LSTM (grey) is nearly flat.

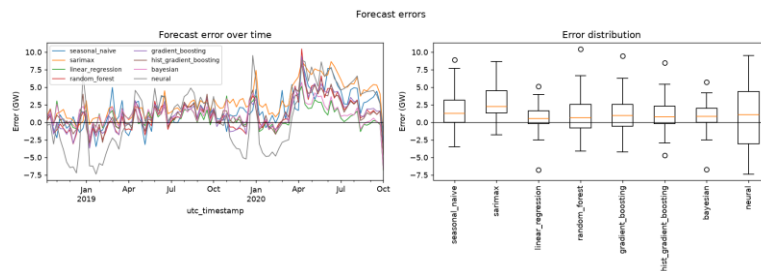


Figure 3. Forecast error over time (left) and its spread (right) for all models. The 2020 drop causes the large positive errors in spring 2020. Linear regression has the tightest error box, the whole SARIMAX distribution sits above zero, and the LSTM's is the widest.

7. Answers to the analysis questions

1. Which models beat the seasonal naive benchmark? Five do, and by a meaningful amount: linear regression by 42%, the Bayesian ridge by 37%, and the three tree models by 16 to 20%. The linear model is the only one with a MASE below 1. SARIMAX is 26% worse than the benchmark, and the LSTM, at 48% worse, is close to simply forecasting the training mean.

2. How was data leakage avoided in the temperature lag features? All temperature lags are backward shifts, so the value for a given week can only come from earlier weeks, never from the week itself or later ones. The same rule applies to the load lags and rolling averages, where the series is shifted back one week before any window is taken. Scaling is fitted on the training data only. The split is strictly chronological. Section 4 gives the detail.

3. Why these differencing orders and this seasonal period for SARIMAX? The seasonal period is 52 because the data are weekly and the dominant cycle is yearly. The stationarity tests agree the level series has no unit root, so the differencing is not there to fix one. The seasonal difference $D = 1$ is included so the model can represent the yearly cycle that the ACF clearly shows at lag 52, even though KPSS flags the differenced series as slightly over-differenced. The regular difference $d = 1$ handles the remaining slow movement, including

the 2020 drop. A second difference was considered and rejected, because it would over-difference a series whose trend is already mild.

4. Do temperature and holiday covariates help, and are they known at the forecast origin? Both help, but differently. Temperature carries a strong signal (+0.69 with heating degree-days), though at weekly resolution it says almost the same thing as the time of year, which the calendar features already cover. Holidays matter most in the Christmas weeks and are fully known in advance, so they are a legitimate input for a real forecast. Future temperature is not. Because the SARIMAX here uses observed test-period temperature, it is a conditional forecast: it answers “given the weather that happened, how good is the model?” rather than “what would we have forecast at the time?”. A real system would need forecast temperature, which carries its own error.

5. Interpretability and complexity of SARIMAX, feature-based, and neural models. They sit on a clear ladder. The linear and Bayesian regressions are the easiest to read: every coefficient has a direct meaning, and the Bayesian version adds honest uncertainty. SARIMAX is also readable, with interpretable orders and free prediction intervals, but its behaviour when the world changes is harder to reason about, as 2020 showed. The tree models are in the middle: importances give intuition, but the model is hundreds of trees and intervals must be added separately. The LSTM is the hardest to interpret, the heaviest to maintain, and here also the least accurate. On this problem the interpretability and accuracy rankings happen to match, so there is no trade-off to weigh.

6. Which model is recommended for operational use? The Bayesian ridge regression, run alongside the seasonal naive as a permanent baseline. On accuracy it is within 9% of the best model. On uncertainty it is the only strong performer with built-in, well-calibrated prediction intervals, which an operator actually needs; the linear model would need extra work to match this. On interpretability its coefficients can be explained to a non-specialist. On maintenance it retrains in milliseconds with nothing to tune. Its weakness is poor extrapolation outside the training range, so it should run with drift monitoring and scheduled retraining. The seasonal naive stays in the loop because any forecast that cannot beat it should trigger a review. SARIMAX is not recommended on this horizon, and the LSTM cannot be justified at this data size on any of the four criteria.

8. Why the results look the way they do

The error plot (Figure 3) explains the ranking better than the table does. Three episodes stand out.

The first is the Christmas and New Year fortnight, when demand collapses. The models with a holiday input, meaning the whole feature-based family and SARIMAX, catch most of the dip. The seasonal naive catches it only when last year’s dip fell in the same week. The LSTM misses it completely.

The second, and the decisive one, is the pandemic demand drop from March 2020, visible in Figure 2 as the actual line falling away from every forecast. The regressions and trees, whose strongest inputs are the recent load lags and rolling averages, adjust within a few weeks because their inputs fall along with the data. The seasonal naive, replaying pre-pandemic

2019, runs high for the remaining six months. SARIMAX does worst of the serious models: tied to the level and shape it learned from 2015 to 2018, it drifts steadily above the actual, which is why its bias of +2.93 GW is by far the largest in the table and its whole error distribution in Figure 3 sits above zero. A structural break inside the test window is close to a worst case for a model that extrapolates a fixed pattern. This is also the direct answer to how the forecasts compare with the real data after the modelling period: methods that reuse recent observed behaviour tracked the real two years well, and the method that extrapolated a fitted pattern did not.

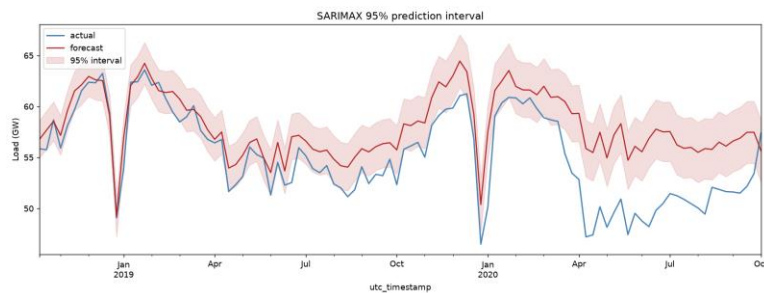


Figure 4. SARIMAX forecast with its 95% prediction interval. The interval is sensible through 2019, then the actual series breaks below the band from April 2020. The pandemic drop is outside anything the model could have learned from its training data.

The third episode concerns the LSTM, whose forecast in Figure 2 is nearly flat at around 56 GW. With 197 training points and a 104-step recursive horizon, the network settled on something close to an average and could not keep the seasonal wave going while feeding on its own output. Small bias (+0.88 GW) together with a very large RMSE is the signature of a forecast that is centred but has no shape. This does not condemn LSTMs in general; it shows that 197 weekly points are far below the amount of data the method needs, answering the data-sufficiency question by experiment rather than by assumption.

The SARIMAX residual ACF (Figure 5) is almost entirely inside the confidence band, so the model fitted the training period well. Its test failure is about the world changing, not a badly fitted model.

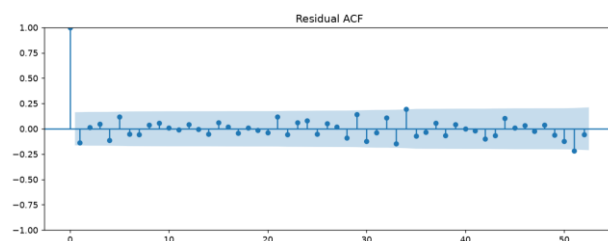


Figure 5. SARIMAX residual autocorrelation. The training-period fit is clean; the test failure comes from the 2020 break, not from unmodelled autocorrelation.

Why did the simplest model win? Three reasons, all traceable to the code. The feature table does most of the work: once lags, rolling averages, seasonal terms, holidays and degree-days exist as columns, what remains is close to a straight-line relationship, and a linear model estimates that with very little variance from 250 rows. The trees, limited to their training

range and predicting in steps, extrapolate slightly worse into the 2020 regime, and with this little data their flexibility mostly adds noise. And the two complex models carry built-in weaknesses: SARIMAX extrapolates a fixed pre-2020 pattern, and the LSTM lacks data. It is the usual bias-variance trade-off at small sample sizes, nothing more.

9. Limitations and future work

Some limits should be stated plainly. Berlin is a single-city stand-in for national temperature. The test period contains a pandemic, a hard but unusual trial: the ranking might tighten in a calmer two years, and a single split cannot say by how much. The LSTM was given weekly data, its hardest setting; the same network on the hourly series, with over 43,000 training points, would be a much fairer test and is the obvious next experiment. The SARIMAX orders followed the recommended starting values rather than a full search.

Future improvements follow directly: rolling-origin evaluation (Hyndman and Athanasopoulos, 2021) to replace the single split with many; conformal intervals to give the linear model the uncertainty it lacks; forecast temperature from a weather provider to turn the conditional SARIMAX into a true operational forecast; an hourly LSTM or a newer design such as N-BEATS (Oreshkin et al., 2020) to test whether neural methods recover with enough data; and averaging the linear, Bayesian and tree forecasts, which would probably shave a little more error at almost no cost.

10. Conclusion

On weekly German electricity demand, a linear regression on a carefully built, leakage-free feature table beat every other method tested, cutting RMSE by 42% against a strong seasonal naive benchmark, with the Bayesian ridge close behind and the better choice operationally because of its built-in uncertainty. The two most complex models both lost to the benchmark, for different and useful reasons: SARIMAX because a fixed fitted pattern extrapolates badly across a structural break, and the LSTM because 197 weekly observations are nowhere near enough for the method. The practical lesson is that with small samples, time spent on features, honest inputs and careful evaluation pays better than time spent on model sophistication.

References

- Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735-1780.
- Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice*. 3rd edn. Melbourne: OTexts.
- Kong, W., Dong, Z.Y., Jia, Y., Hill, D.J., Xu, Y. and Zhang, Y. (2019) 'Short-term residential load forecasting based on LSTM recurrent neural network', *IEEE Transactions on Smart Grid*, 10(1), pp. 841-851.
- Marino, D.L., Amarasinghe, K. and Manic, M. (2016) 'Building energy load forecasting using deep neural networks', *Proceedings of IECON 2016*, pp. 7046-7051.